

Software Requirements Specification (SRS) for Fully Flexible Credit System (FFCS)

*Kaivlya Shah 24BCA7036
Ayushman Bhattacharyya 24BCA7801
Harshdeepsingh Sardar 24BCE7196
Akshat Jain 24BCE7908*

17/12/2025

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Initial Draft	Team	Initial Draft	17/12/2025

Review & Approval

Approving Party	Version Approved	Signature	Date
Professor	N/A		

Contents

1. Introduction
2. General Description
3. Functional Requirements
4. Interface Requirements
5. Performance Requirements
6. Other non-functional attributes
7. Operational Scenarios
8. Preliminary Use Case Models and Sequence Diagrams
9. Updated Schedule
10. Updated Budget
11. Appendices

1. Introduction

The purpose of this document is to define and describe the requirements of the Fully Flexible Credit System (FFCS) project and to spell out the system's functionality and its constraints.

Scope of this Document

This SRS defines the requirements for an online system that allows university students to select courses, teachers, and time slots based on their assigned subjects and available credits. It covers the system's core features, including time clash checks, seat limit enforcement, secure login, and dynamic timetable generation. It also covers the interface for the FFCS organizer to manage course sections, seat limits, and overall system configuration.

Overview

The FFCS product is a web-based system designed to manage course registration and slot allocation in a flexible credit system environment. It will feature two main interfaces: the Student Interface and the Organizer Interface.

2. General Description

2.1 Product Functions

The product should make the subject/slot selection process easy and flexible for the students and provide powerful configuration tools for the organizers.

2.2 Similar System Information

The product is a web-based registration system. Its competitive edge will be its focus on performance, handling 2000-3000 concurrent requests, and its built-in flexibility for university-specific timetable definitions.

2.3 User Characteristics

- **Students:** Users who need basic computer literacy to navigate a web-based interface. They require secure login and an intuitive interface for course selection.
- **Organizer (Administrator):** University staff requiring a deep understanding of

academic rules to configure and manage the system.

2.4 User Problem Statement

The users' current course registration system is likely slow, inflexible, or inefficient. It results in scheduling conflicts and difficulty for students to get preferred slots. The FFCS must provide a solution that is fast, flexible, and capable of managing heavy concurrent load.

2.5 User Objectives

The user wants a system that will allow students to select teachers and time slots for subjects. The program must facilitate the speed and ease of input, provide conflict checking, and securely store registration data.

2.6 General Constraints

Constraints include:

- The system must handle **2000-3000 simultaneous requests** efficiently to ensure a smooth user experience during peak registration times.
- The system must enforce the university's credit and subject rules.
- The system must utilize a secure login mechanism (Registration Number, Password, and CAPTCHA).

The platform must be web-based and accessible via standard browsers.

3. Functional Requirements

The system must satisfy the following core functions for both the Student and Organizer interfaces:

ID	Requirement	Interface	Criticality	Dependencies
FR-O-1	Organizer Timetable Definition: Define and name the timetable structure (days and slots) unique to the university.	Organizer	Very High	None

FR-O-2	Slot Timing Assignment: Assign specific start/end timings to each defined slot name (e.g., Slot A: Mon 9:00-9:50).	Organizer	Very High	FR-O-1
FR-O-3	Seat Limit Management: Set the maximum number of seats for each teacher/slot combination.	Organizer	Very High	FR-O-1, FR-O-2
FR-O-4	Course Management: Add, modify, and delete course offerings, teachers, and time slots.	Organizer	High	FR-O-1, FR-O-2
FR-O-5	Timetable Variant Definition: Define separate sets of slot timings based on user group (Day Boarder/Hosteller)	Organizer	Very High	FR-O-2
FR-S-1	Secure Login: Validate student credentials (Reg No., Password, CAPTCHA).	Student	Very High	None
FR-S-2	Credit Display: Display student's current subjects and available credit capacity upon login.	Student	Very High	FR-S-1
FR-S-3	Sorted Slot Selection View: View available slots and teachers, sorted in alphabetical and ascending order.	Student	High	FR-S-2, FR-O-3
FR-S-4	Clash Check and Visual Highlighting: Check for and display clash messages. The system must visually highlight clashing slots in the interface.	Student	Very High	FR-S-6
FR-S-5	Seat Limit Enforcement: Prevent registration if the teacher's seat limit (FR-O-3) is reached.	Student	High	FR-O-3

FR-S-6	Dynamic Timetable Update: Update the weekly timetable upon each successful subject/slot selection.	Student	Very High	FR-S-4, FR-O-2
FR-S-7	Timetable View: Allow students to view their current, cumulative weekly timetable at any time.	Student	High	FR-S-6
FR-S-8	New Subject Selection: Allow selection of new subjects if credits permit and university allows.	Student	High	FR-S-2
FR-S-9	Variant Application: Automatically apply the correct timing set (Day Boarder/Hosteller) for clash checks and display.	Student	Very High	FR-S-4, FR-S-6, FR-O-5

4. Interface Requirements

4.1 User Interfaces

- **GUI:** There will be two primary web-based Graphical User Interfaces (GUIs):
 - **Student Interface:** For course selection and timetable viewing.
 - **Organizer Interface:** For system configuration, including timetable (FR-O-1, FR-O-2) and seat limit management (FR-O-3).
- **CLI:** There is no command line interface.

4.2 Hardware Interfaces

The program is web-based and will utilize the host server's hard disk, memory, and processor. Access to these resources will be managed by the operating system and web server software.

4.3 Communications Interfaces

The system requires standard network communication (TCP/IP) for client-server interaction via the internet/university network.

4.4 Software Interfaces

The FFCS system must interface with:

- **University Database/System:** To fetch student registration details, course data, and credit limits.

CAPTCHA Service: For integration into the student login process (FR-S-1).

5. Performance Requirements

The system's performance is critical due to high concurrent usage.

- **Concurrency:** The system must reliably handle **2000-3000 concurrent user requests** during the peak registration period.
- **Response Time:** The login process and all critical selection operations (slot selection/clash check) should complete within **3 seconds** under peak load.
- **Throughput:** The system should be capable of processing a high volume of registration transactions per minute to avoid user delays.

System Requirements: As a web-based application, requirements exist for the server infrastructure (to be defined separately), but client-side requirements are minimal (standard internet-capable device/browser).

6. Other non-functional attributes

6.1 Security

- Student login must include a CAPTCHA mechanism.
- Access to the Organizer interface must be restricted to privileged credentials.
- Sensitive student and course data must be encrypted where necessary.

6.2 Binary Compatibility

Compatible with any computer running a modern web browser (Chrome, Firefox, Edge, Safari).

6.3 Reliability

- Maintain data integrity regarding seat counts and credit usage to prevent over-registration.
- Backups will be made regularly for quick restoration.

6.4 Maintainability

Documented and structured for maintenance by any developer proficient in the chosen tech stack.

6.5 Portability

Accessible from any standard computing device with internet access.

6.6 Extensibility

Architecture must be flexible to allow for quality of life changes and core feature updates with minimal disruption.

6.7 Reusability

Designed for re-use across multiple registration cycles with minimal configuration changes.

7. Operational Scenarios

Scenario A: Student Login and View

The student navigates to the FFCS website, enters login credentials (including CAPTCHA), and the system validates them. Upon success, the system displays the student's assigned subjects and available credit capacity.

Scenario B: Student Course Selection and Clash Check

The student selects a subject; the system displays available teachers and slots **sorted alphabetically**. The student provisionally selects a slot. If a conflict exists with the existing timetable, the system **visually highlights the clashing slots** and displays the clash error message.

Scenario C: Student Final Registration and Timetable Update

The student confirms registration for a slot. The system performs a final check (clash, seat, credit limit). If all checks pass, the registration is confirmed, and the system dynamically updates the student's weekly timetable (FR-S-6, FR-S-7).

Scenario D: Organizer Initial Timetable Setup

The Organizer logs in and navigates to the 'Timetable Setup' section. The Organizer defines university-specific slot names (e.g., A, B1, T2) and assigns their specific start/end timings on different days (FR-O-1, FR-O-2). This structure is stored for use in all course assignments and student clash checks.

Scenario E: Database Maintenance (Organizer)

The Organizer may want to alter or delete information (e.g., remove an old course or change a teacher's assigned section). They will need administrative access to manage the core data.

8. Preliminary Use Case Models and Sequence Diagrams

This section presents a list of the fundamental sequence diagrams and use cases that satisfy the system's requirements. The purpose is to provide an alternative, "structural" view of the requirements stated above and how they might be satisfied in the system.

8.1 Use Case Model

- **Actors:** Student, FFCS Organizer, University Database.
- **Key Use Cases:** Login, View Available Slots, Check for Clash, Register for Slot, View Timetable, Define Timetable Structure, Set Seat Limits, Manage Courses.

8.2 Sequence Diagrams

Sequence diagrams would illustrate the flow for key processes like:

- Student Login and Validation.
- Slot Selection, Clash Check, and Registration Confirmation.

9. Updated Schedule

To be given by the professor.

10. Updated Budget

To Be Decided

11. Appendices

11.1 Definitions, Acronyms, Abbreviations

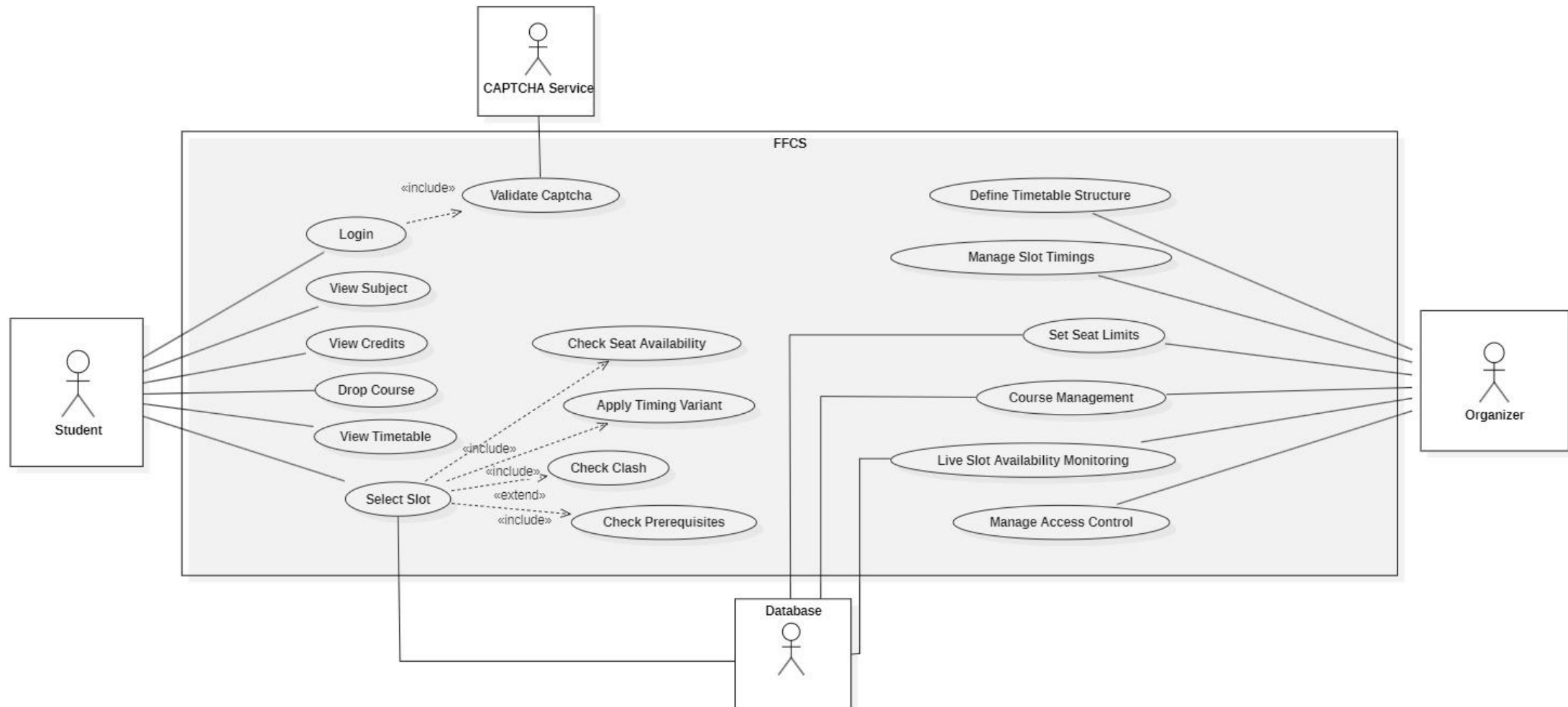
- **FFCS:** Fully Flexible Credit System

- **FR-X-Y:** Functional Requirement Identifier
- **Reg No.:** University Registration Number

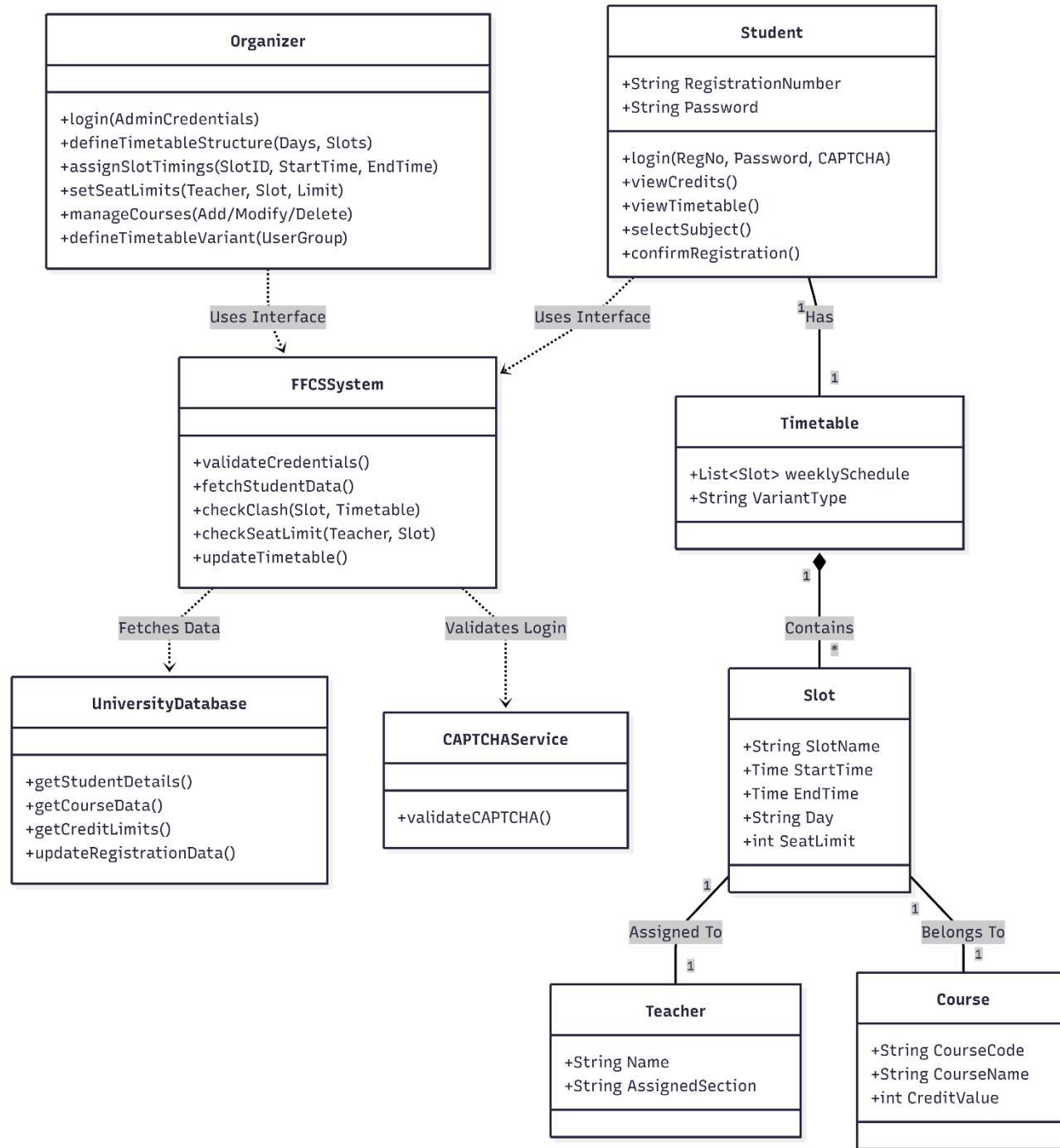
Tasks to do

- 1) Remove the reference number superscript numbers from the document (done)**
- 2) Format the document to make it better looking [(kind of done) just left the highlighted part]**
- 3) Extra features: (yet to be decided)**
 - Ability for the organizer to see the availability of slots live**
 - Ability for the organizer to manage access control**
 - The system shall allow students to drop/withdraw from a previously registered course, updating the seat count immediately**
 - Checking of prerequisites and antirequisites**

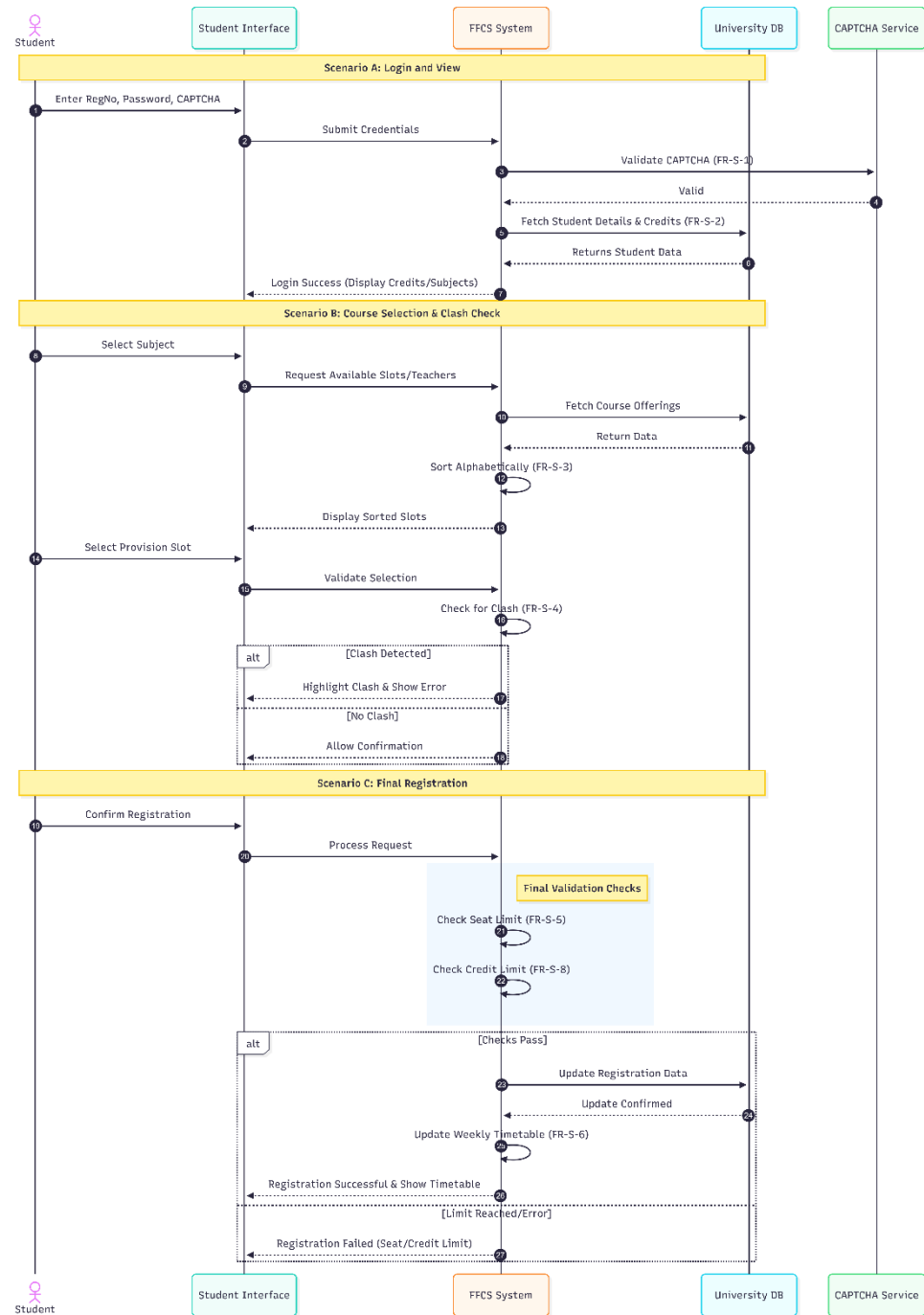
The Use Case Diagram



The Class Diagram



Sequence Diagram: Student Registration



Sequence Diagram: Organizer Configuration

